3.1. OBJETOS PREDEFINIDOS DEL LENGUAJE

- **Resultado de Aprendizaje (RA): RA3:** *Escribe código, identificando y aplicando las funcionalidades aportadas por los objetos predefinidos del lenguaje*.
- **Criterio de Evaluación (CE): CE 3.a:** *Se han identificado los objetos predefinidos del lenguaje*.
- **Ponderación dentro del RA3: 6,67% del RA3 | 1% sobre la calificación final del módulo**).

3.1.1. Arquitectura de objetos en el cliente: Globales nativos frente a objetos del host

En el desarrollo web en entorno cliente, JavaScript ejecuta su código dentro de un contexto provisto por el navegador. Es fundamental distinguir entre los objetos que forman parte de la especificación universal de **ECMAScript (objetos nativos)** y aquellos creados por el entorno del navegador **(objetos del host / BOM y DOM)**:



1. **Objetos nativos del lenguaje:** Definidos por la especificación ECMA-262. Están disponibles en cualquier entorno de JavaScript (navegador, Node.js, etc.). En este criterio nos centramos en los objetos de cálculo, gestión temporal y utilidades primitivas: Math, Date, String y Number.
2. **Objetos del host (BOM / DOM):** Son suministrados por el navegador web y cuelgan de window. Permiten controlar la ventana, el historial, la pantalla y el árbol de nodos del documento HTML. Su estudio comienza a partir del siguiente apartado 3.2.

3.1.2. El objeto estático Math

El objeto Math es un contenedor nativo de constantes matemáticas y funciones de cálculo numérico. **No es una función constructora**; carece de método constructor y, por tanto, **no se puede instanciar mediante new Math()**. Todos sus miembros son estáticos.

Constantes matemáticas principales

- **Math.PI:** Relación aproximada entre el perímetro de una circunferencia y su diámetro (~3.141592653589793).
- **Math.E:** Constante de Euler o base de los logaritmos naturales (~ 2.718281828459045).
- **Math.LN10 / Math.LN2:** Logaritmo natural de 10 (~2.302) y de 2 (~0.693).
- **Math.SQRT2:** Raíz cuadrada de 2 (~1.414).

Métodos de redondeo y aproximación

- **Math.round(x):** Redondeo aritmético al entero más próximo. Si la parte fraccionaria es ≥ 0.5, aproxima hacia el entero superior más cercano.

```
console.log(Math.round(4.49)); // 4

console.log(Math.round(4.50)); // 5

console.log(Math.round(-3.50)); // -3 (en JavaScript, -3.5 se redondea hacia arriba en la recta real hacia -3)
```

- **Math.floor(x):** Devuelve el mayor entero menor o igual al parámetro (redondeo hacia abajo).

```
console.log(Math.floor(8.99)); // 8

console.log(Math.floor(-8.01)); // -9 (se aleja de cero hacia el entero inferior)
```

- **Math.ceil(x):** Devuelve el menor entero mayor o igual al parámetro (redondeo hacia arriba).

```
console.log(Math.ceil(3.01)); // 4

console.log(Math.ceil(-3.99)); // -3
```

- **Math.trunc(x):** Trunca el número descartando la parte decimal sin aplicar redondeo.

```
console.log(Math.trunc(5.87));  // 5

console.log(Math.trunc(-5.87)); // -5 (a diferencia de Math.floor, no baja a -6)
```

Métodos algebraicos, trigonométricos y comparativos

- **Math.abs(x):** Valor absoluto de una magnitud.

```
console.log(Math.abs(-45)); // 45
```

- **Math.sqrt(x) y Math.cbrt(x):** Raíz cuadrada y cúbica respectivamente.

```
console.log(Math.sqrt(64));  // 8

console.log(Math.sqrt(-9));  // NaN (no opera con números complejos)

console.log(Math.cbrt(-27)); // -3 (la raíz cúbica sí admite radicando negativo)
```

- **Math.pow(base, exponente):** Exponenciación de una base.

```
console.log(Math.pow(2, 4)); // 16

console.log(2 ** 4);         // 16 (operador de exponenciación, sintaxis moderna preferida)
```

- **Math.min(...valores) y Math.max(...valores):** Localizan el extremo mínimo o máximo en una lista de argumentos.

```
console.log(Math.min(10, 2, -5, 18)); // -5

console.log(Math.max(10, 2, -5, 18)); // 18

console.log(Math.min()); // Infinity

console.log(Math.max()); // -Infinity
```

- **Math.sin(rad), Math.cos(rad), Math.tan(rad):** Funciones trigonométricas estándar en radianes.

```
console.log(Math.sin(Math.PI / 2)); // 1 (seno de 90 grados)

console.log(Math.round(Math.cos(Math.PI / 2))); // 0 (se usa round por la imprecisión de coma flotante IEEE 754)
```

- **Math.hypot(...valores):** Retorna la raíz cuadrada de la suma de los cuadrados de sus argumentos (teorema de Pitágoras).

```
console.log(Math.hypot(3, 4)); // 5 (sqrt(3^2 + 4^2) = 5)
```

Generación pseudoaleatoria con Math.random()

Math.random() devuelve un número decimal pseudoaleatorio en el rango semiabierto $[0, 1)$.

Para obtener un entero aleatorio dentro de un intervalo cerrado $[min, max]$, la fórmula matemática estándar es:

```
const min = 1;

const max = 6;
```

```
// Genera un número entero entre min y max (ambos inclusive)

const tiradaDado = Math.floor(Math.random() * (max - min + 1)) +
min;

console.log(tiradaDado); // Entero en [1, 6]
```

Aleatoriedad y Seguridad Básica: El método moderno y seguro: `window.crypto`

Son síncronos, rápidos y están disponibles directamente en el objeto global crypto.

- **crypto.randomUUID()**
  - **Qué hace:** Genera un identificador único universal (UUID v4) de 36 caracteres de forma criptográficamente segura (ej: 7b3b4f62-63b1-4f8e-94df-60e0a5160893).
  - **Uso común:** Creación de IDs de sesión, tokens de recuperación y claves primarias en el cliente.
  - *Nota:* Solo funciona en entornos seguros (HTTPS o localhost).

```
// Generar un ID seguro para un objeto o sesión const
idPedido = crypto.randomUUID(); console.log(idPedido); //
Ejemplo de salida: "2d1b7484-88d4-4bb8-8d4e-128a3f1234bc"
```

- **crypto.getRandomValues(typedArray)**
  - **Qué hace:** Llena un array de enteros de tipo fijo (como Uint8Array o Uint32Array) con valores pseudoaleatorios fuertes basados en entropía del hardware.
  - **Uso común:** Generación de claves criptográficas en bruto, vectores de inicialización (IV) o números aleatorios a medida.

```
// Generar un bloque de 16 bytes aleatorios (útil para
tokens crudos o Salts) const bufferBytes = new
Uint8Array(16); crypto.getRandomValues(bufferBytes);
console.log(bufferBytes); // Ejemplo de salida:
Uint8Array(16) [23, 142, 201, 8, 44, ...]
```

- **crypto.randomInt(min, max)**
  - **Qué hace:** Devuelve directamente un entero aleatorio seguro dentro de un rango determinado (inclusivo en min, exclusivo en max).
  - **Uso común:** Reemplazo directo y seguro para la fórmula clásica de Math.random(). (Originalmente del backend de Node.js).

```
// Simular la tirada de un dado de 6 caras de forma segura (rango 1 a 6) const tiradaDado = crypto.randomInt(1, 7); console.log(tiradaDado); // Devuelve: 1, 2, 3, 4, 5 o 6
```

- **Navegador web:** crypto.randomInt **no existe** de forma nativa en el objeto window.crypto del navegador. Si tu código se va a ejecutar en el cliente (browser), deberás usar una solución equivalente exacto a **Math.random()** con getRandomValues().

```
function cryptoRandomFloat() {

  // Aquí estás creando un "contenedor" para almacenar el número.

  const buffer = new Uint32Array(1);

  /*Uint32Array: Significa Unsigned Integer 32-bit Array (Array de Enteros de 32 bits Sin Signo). Es un tipo de dato especial en JavaScript que solo puede almacenar números enteros positivos (sin decimales ni signos negativos).*/

  window.crypto.getRandomValues(buffer);

  /* Dividimos el entero obtenido entre el valor máximo.
  Al ser de 32 bits, puede almacenar cualquier número entero desde el 0 hasta el 4.294.967.295 (2³² - 1).*/

  return buffer / 4294967296;

}

console.log(cryptoRandomFloat()); // Devuelve algo como 0.4765625... de forma 100% segura
```

De esta forma se simula Math.random() obteniendo un número decimal continuo entre 0 y 1 (con todos los decimales posibles que quepan en la memoria)

Característica	Enfoque Math.random()	Enfoque crypto
Origen	Motor de JavaScript (Algoritmo matemático puro)	Entropía del Sistema Operativo (Hardware)
Rendimiento	**Ultra rápido**. Ideal para juegos, simulaciones y lógica visual.	Más lento debido a las llamadas al sistema.
Predicibilidad	**Predecible**. Si alguien conoce el estado interno, puede adivinar los siguientes números.	**Impredecible**. Imposible de hackear o anticipar.
Caso de uso	Dados, barajar cartas visualmente, partículas, IDs temporales no críticos.	Contraseñas, tokens de sesión, claves, sorteos de dinero real.

Un error clásico de los programadores junior es emplear Math.random() para generar contraseñas temporales, números de pedido o claves de recuperación. Al ser un algoritmo pseudoaleatorio determinista predecible, los estándares de auditoría de seguridad rechazan su uso. En producción se exige el empleo de identificadores universales seguros generados de forma nativa por el navegador mediante window.crypto.randomUUID():

```
const idPedido = window.crypto.randomUUID();

console.log(idPedido); // "7b3b4f62-63b1-4f8e-94df-60e0a5160893"
```

3.1.3. Manipulación del tiempo y fechas con Date

El objeto predefinido Date administra fechas y horas en el cliente. Su valor numérico interno representa los **milisegundos transcurridos desde el 1 de enero de 1970 a las 00:00:00 UTC** (época UNIX).

Constructores del objeto Date

Existen cuatro modalidades sintácticas para instanciar un objeto Date:

```
// 1. Momento exacto del sistema en el instante de ejecución

const fechaActual = new Date();

// 2. Mediante milisegundos transcurridos desde el origen UNIX

const fechaCero = new Date(0); // 01/01/1970 00:00:00 UTC

// 3. Cadena estandarizada (formato ISO 8601:
YYYY-MM-DDTHH:mm:ss.sssZ)

const fechaCadena = new Date("2026-10-25T14:30:00Z");
```

```
// 4. Parámetros independientes: año, mes (0-11), día (1-31), hora, min, seg, ms

// ATENCIÓN: El mes está indexado en base cero (0 = Enero, 9 = Octubre)

const sesionEvaluacion = new Date(2026, 9, 25, 10, 15, 0);
```

Métodos de lectura equivalentes en tiempo universal (UTC)

Permiten inspeccionar el instante sin la desviación local:

```
const f = new Date(2026, 9, 25, 10, 15, 0);

console.log(f.getUTCFullYear());

console.log(f.getUTCMonth());

console.log(f.getUTCDate());

console.log(f.getUTCHours());
```

Métodos de serialización y marcas de tiempo

- **Date.now():** Retorna la marca temporal actual en milisegundos sin instanciar un objeto en memoria (método estático de alto rendimiento).

  ```
  const inicio = Date.now();

  const tipo = typeof(inicio); //number

  // Operación a medir...

  const duracion = Date.now() - inicio;

  console.log(`Duración: ${duracion} ms`);
  ```

- **toISOString():** Cadena bajo el estándar universal ISO 8601 (YYYY-MM-DDTHH:mm:ss.sssZ), obligatoria para APIs y bases de datos.

  ```
  console.log(new Date().toISOString()); // Formato: "2026-10-25T12:00:00.000Z”
  ```

- **toUTCString():** Formato de fecha estándar RFC 1123/7231, requerido en la directiva expires para la creación de cookies HTTP en el BOM.

  ```
  console.log(new Date().toUTCString()); // Formato: "Sun, 25 Oct 2026 12:00:00 GMT"
  ```

- **toDateString() / toTimeString():** Representaciones textuales legibles reducidas.

```
console.log(new Date().toDateString()); // Formato: "Sun Oct 25 2026"
```

En entornos reales, los clientes web nunca deben almacenar ni intercambiar fechas en formato local dependiente de la máquina del usuario. Los servidores transmiten marcas temporales en formato estandarizado **UTC / ISO 8601** con sufijo 'Z' (ej. 2026-10-25T12:00:00.000Z). El desarrollador de cliente utiliza el objeto Date exclusivamente para recibir ese valor y representarlo en la interfaz adaptado a la zona horaria del usuario.

3.1.4. Objetos envolventes (*Wrappers*) de tipos primitivos

JavaScript define los datos elementales como **tipos primitivos** (string, number, boolean, null, undefined). A diferencia de lenguajes orientados a objetos puros, en JavaScript los primitivos no son objetos ni contienen métodos propios en memoria.

Para permitir la ejecución de instrucciones como "texto".toUpperCase(), el motor aplica el proceso de **autoboxing**:

1. Envuelve temporalmente el valor primitivo en una instancia del objeto correspondiente (String, Number, Boolean).
2. Invoca el método solicitado sobre la instancia temporal.
3. Destruye inmediatamente el objeto temporal devolviendo el resultado primitivo.

Regla técnica fundamental: Nunca utilices new String(), new Number() ni new Boolean(). Instanciar primitivos con new crea objetos pesados en memoria, degradando el rendimiento y provocando errores graves de comparación estricta (**===**):

```
const primitivo = 25;

const objetoEnvolvente = new Number(25);

console.log(typeof primitivo);        // "number"

console.log(typeof objetoEnvolvente); // "object"

console.log(primitivo == objetoEnvolvente);  // true (coerción implícita)

console.log(primitivo === objetoEnvolvente); // false (fallo de identidad por disparidad de tipos)
```

Métodos del objeto envolvente String

- **slice(inicio, fin):** Extrae una sección de la cadena. Admite índices negativos contando desde el final.

```
const modulo = "Desarrollo Web en Entorno Cliente";

console.log(modulo.slice(0, 10)); // "Desarrollo"

console.log(modulo.slice(-7));    // "Cliente"
```

- **substring(inicio, fin):** Extrae caracteres entre dos posiciones. Si inicio > fin, invierte automáticamente los parámetros. Trata índices negativos como 0.

```
console.log(modulo.substring(10, 0)); // "Desarrollo"
```

- **replace(patron, nuevoTexto) y replaceAll(patron, nuevoTexto):** Reemplazan coincidencias.

```
const texto = "Java y JavaScript comparten sintaxis con
Java";

console.log(texto.replace("Java", "Kotlin")); // Solo
reemplaza la primera coincidencia

console.log(texto.replaceAll("Java", "ECMA")); // Reemplaza
todas las apariciones
```

- **trim(), trimStart() y trimEnd():** Eliminan espacios en blanco en los extremos.

```
const email = "   usuario@ieskursaal.es \n ";

console.log(`'${email.trim()}'`); //
"'usuario@ieskursaal.es'"
```

- **padStart(longitud, relleno) y padEnd(longitud, relleno):** Rellenan la cadena hasta alcanzar la longitud deseada.

```
const idFactura = "89";

console.log(idFactura.padStart(6, "0")); // "000089"
```

- **charAt(indice):** Devuelve el carácter ubicado en la posición indicada.

```
console.log("DWEC".charAt(2)); // "E”
```

- **repeat(veces):** Devuelve una nueva cadena con el número de copias consecutivas especificado.

```
console.log("=".repeat(10)); // "=========="
```

- **split(delimitador, limite):** Divide la cadena y retorna un array

```
const csv = "Juan,Carlos,Moreno";

console.log(csv.split(",")); // ["Juan", "Carlos", "Moreno"]
```

- **includes(subcadena) y startsWith(subcadena):** Comprobaciones booleanas de presencia textual.

```
console.log("https://ieskursaal.es".startsWith("https://"));
// true

console.log("algeciras".includes("geci")); // true
```

Métodos del objeto envolvente Number

- **toFixed(decimales):** Formatea el número a cadena redondeando al número de decimales fijado.

```
const precio = 19.956;

console.log(precio.toFixed(2)); // "19.96" (devuelve string
para preservar ceros finales)
```

- **toPrecision(cifras):** Formatea el número a una cantidad fija de dígitos significativos.

```
const val = 9.99;

console.log(val.toPrecision(1)); // "1e+1" Redondea a 10. Se
fuerza notación científica para usar solo un dígito (1).

console.log(val.toPrecision(2)); // "10" Redondea a 10. Usa
dos dígitos (1 y 0).

console.log(val.toPrecision(3)); // "9.99" Muestra el número
original exacto.

console.log(val.toPrecision(4));     "9.990"    Añade un cero
al final para cumplir con los 4 dígitos exigidos.
```

- **Number.parseInt(cadena, base) y Number.parseFloat(cadena):** Conversión de cadenas a números analizando los caracteres iniciales.

```
console.log(Number.parseInt("48px", 10)); // 48

console.log(Number.parseInt("101", 2));    // 5 (interpretado
en base binaria)

console.log(Number.parseFloat("12.50 €")); // 12.5
```

- **Number.isNaN(valor) frente a la función global clásica isNaN(valor):**

```
// La función global clásica aplica coerción implícita
indeseada:
```

```
console.log(isNaN("Algeciras")); // true (porque "Algeciras"
convertido a número da NaN)

console.log(isNaN("   "));       // false (¡peligro!
convierte espacios en blanco a 0)

// El método de ES6 Number.isNaN es estricto: solo evalúa a
true si el tipo es Number Y su valor es NaN:

console.log(Number.isNaN("Algeciras")); // false (es de tipo
string, no el valor numérico NaN)

console.log(Number.isNaN(NaN));          // true

console.log(Number.isNaN(0 / 0));        // true
```

- **Number.isFinite(valor) frente a la función global isFinite(valor):**

```
console.log(isFinite("50"));        // true (convierte
implícitamente el texto a número)

console.log(Number.isFinite("50")); // false (evaluación
estricta sin coerción)

console.log(Number.isFinite(50));   // true

console.log(Number.isFinite(1 / 0)); // false (Infinity no es
finito)
```

- **Number.isInteger(valor):** Determina si el valor es un número entero exacto.

```
console.log(Number.isInteger(10));   // true

console.log(Number.isInteger(10.5)); // false

console.log(Number.isInteger("10")); // false
```

3.1.5. Constantes numéricas especiales

- **Infinity y -Infinity:** Representan desbordamientos cuando una operación supera el límite superior ($1.7976931348623157 \cdot 10^{308}$) o inferior del estándar IEEE 754.

```
console.log(1.8e308);  // Infinity

console.log(-1.8e308); // -Infinity

console.log(1 / 0);    // Infinity
```

- **NaN (*Not a Number*):** Representa un valor que no es numérico resultante de operaciones indefinidas ($0/0$, Math.sqrt(-1)).
- **undefined:** Indica que una variable declarada carece de asignación en memoria.

```
let variableSinValor;

if (typeof variableSinValor === "undefined") {]

  console.log("Variable sin inicializar en memoria.");

}
```

3.1.6. El problema histórico: La fragmentación del objeto global

En el desarrollo web profesional se comparte lógica entre cliente y servidor (arquitecturas *isomórficas* o *SSR* como Next.js, Nuxt o Node.js). En Node.js o en un *Web Worker* del navegador, window no existe y su invocación arroja un error crítico (ReferenceError).

- **La solución estándar (ES2020):** Se introdujo el objeto predefinido **globalThis**, que unifica el acceso al objeto global independientemente de dónde se ejecute el código (apunta a window en el hilo principal del navegador, a self en un Web Worker y a global en entornos de servidor).

  ```
  // Mala práctica tradicional (falla en workers o SSR):

  const temporizador_w = window.setTimeout(() => {

    console.log("Ejecutado sobre el objeto global universal.");

  }, 1000);


  // Buena práctica estándar multiplataforma:

  const temporizador_g = globalThis.setTimeout(() => {

    console.log("Ejecutado sobre el objeto global universal.");

  }, 1000);
  ```

- **Ejemplo práctico donde se usa globalThis:** Crear una variable global "Universal"

  Imagina que estás escribiendo un script de analítica (como Google Analytics) o una librería que se va a usar tanto en el servidor (Node.js/SSR) como en la web. Necesitas guardar una configuración en un lugar donde todo el programa pueda leerla.

  Si intentas asignarla a la antigua, romperás algún entorno:

  ```
  // Error en Node.js (window no existe)
  ```

```
window.miConfiguracion = { apiKey: "123" };

// Error en el Navegador (global no existe)

global.miConfiguracion = { apiKey: "123" };
```

La solución con globalThis:

```
//  Perfecto y seguro en Navegador, Node.js, Next.js (SSR) y
Workers
globalThis.miConfiguracion = { apiKey: "123" };

// En cualquier otro archivo de tu aplicación, puedes leerlo con
total seguridad:

console.log(globalThis.miConfiguracion.apiKey);
```

Actividades

Actividad 3.1.1. Descifra los Rangos de Math.random()

Analiza detalladamente el código de las siguientes 5 funciones de JavaScript. Para cada una de ellas, debes responder a dos preguntas observando su lógica matemática y las operaciones de redondeo empleadas:

1. ¿Cuál es el rango exacto de números que puede generar? (Especificar el intervalo cerrado [min, max]).
2. ¿Cuántas cifras decimales tiene el resultado? (0 para enteros, 1, 2 o 3).

```
// ========================================================
// BLOQUE DE FUNCIONES(MATH.RANDOM)
// ========================================================

function funcionA() {
  const amplitud = 10; // (20 - 10)
  const enteroBase = Math.floor(Math.random() * (amplitud + 1)) + 10;
  return enteroBase;
}

function funcionB() {
  const amplitud = 10;
  const enteroBase = Math.floor(Math.random() * (amplitud + 1));
  return Number((enteroBase / 10).toFixed(1));
}

function funcionC() {
```

```
  const minEscalado = 500;
  const maxEscalado = 1500;
  const enteroBase = Math.floor(Math.random() * (maxEscalado -
minEscalado + 1)) + minEscalado;
  return Number((enteroBase / 100).toFixed(2));
}

function funcionD() {
  const amplitud = 5;
  const enteroBase = Math.floor(Math.random() * (amplitud + 1)) + 1;
  return enteroBase;
}

function funcionE() {
  const minEscalado = -1000;
  const maxEscalado = 1000;
  const enteroBase = Math.floor(Math.random() * (maxEscalado -
minEscalado + 1)) + minEscalado;
  return Number((enteroBase / 1000).toFixed(3));
}

const numeroA = () => Math.floor(Math.random() * 101);

const numeroB = () => Math.floor(Math.random() * 2);

function F() {
  // Generamos un entero entre 0 y 2^53 (el límite seguro de JS)
  const maxGranEntero = 9007199254740991;
  const enteroAleatorio = Math.floor(Math.random() * (maxGranEntero +
1));
  return enteroAleatorio / maxGranEntero;
}
```

Actividad 3.1.2. Generar funciones análogas a las anteriores usando la función cryptoRandomFloat

Actividad 3.1.2. Función random genérica

Fusionar las funciones del ejercicio anterior en una única función.

- **Instrucciones:**
    1. Diseña y documenta una función única llamada generarAleatorio que reciba los siguientes parámetros opcionales: min, max, decimales.
    2. Debe incluir control de errores: si el usuario pasa un min mayor que el max, la función debe lanzar un error explícito (throw new Error).
    3. Debe devolver un tipo de dato Number.
    4. Demostración de casos de uso:

```
console.log("Entero [0, 100]:", generarAleatorio(0,100));
console.log("Coordenada GPS [0.125, 0.875] de 3 cifras:",
generarAleatorio(0.125, 0.875, 3));
```

5. Añade un cuarto parámetro a la función (seguro) para que genere el número usando la función cryptoRandomFloat()

```
console.log("Coordenada GPS [0.125, 0.875] de 3 cifras (Segura):",
generarAleatorio(0.125, 0.875, 3, “segura”));
```

Actividad 3.1.3: Validador de Facturación y Conversión Numérica

Crea una página web con un formulario simple donde el usuario introduzca el precio sin IVA de un artículo (que puede venir con comas o puntos, o caracteres accidentales al final como €) y el porcentaje de IVA aplicable (por defecto 21).

El script debe:

1. Extraer el valor numérico válido descartando espacios en blanco mediante trim() y reemplazando comas por puntos.
2. Verificar estrictamente mediante Number.isNaN() y Number.isFinite() que el importe sea una cifra positiva real.
3. Calcular la cuota de IVA y el precio total de venta al público (PVP).
4. Mostrar en pantalla el resultado redondeado a dos cifras decimales con toFixed(2).

Actividad 3.1.4: Generador de Matrículas y Cripto-Identificadores

Desarrolla una aplicación web con dos botones:

- **"Generar Matrícula Española (DGT)":** Debe generar una matrícula válida compuesta por 4 números aleatorios (rellenos con ceros a la izquierda mediante padStart(4, "0") si el número es menor a 1000) y 3 letras mayúsculas aleatorias consonantes (descartando vocales para seguir la norma de la DGT: sin A, E, I, O, U, ni Ñ ni Q).